

some of the root flaws in your system, you'll be unable to do new product development because you're drowning in the complexity."

#### - Bad Reasons For Microservices

>Maintaining a bunch of loose microservices is not easier than maintaining a monolith. There might be an argument for moving from a distributed monolith to microservices, provided that the organizational problems that led to a distributed monolith have been addressed. A shift to microservices is likely to make things worse before it makes them better. If things are already bad, that's going to make life difficult.

#### Process for proposing a separate database

The process for proposing a separate database has moved to the required approvals section of our handbook.

---

## SECTION: engineering/infrastructure-platforms/data-access/database-framework/doc/container-registry/index.md

This page is meant to track the discussion of different database design approaches for the Container Registry.

#### Background and reading material

Discussion of the database schema (model 1 below) along with queries, estimated database size and expected query rates.

#### API specs

Top-level epic for container registry's garbage collection with zero downtime

#### Deduplication ratios

A docker manifest describes how a docker image comprises of multiple layers. A manifest can be identified by the layers it references and as such it can be thought of as unique throughout the registry. Multiple repositories reference the same manifest.

Below we discuss different approaches to modeling that. Model 1 aims to deduplicate manifests, so that there is only ever one record for the same manifest and repositories merely reference manifests. Model 2 does not deduplicate manifests this way. Here, a manifest always belongs to a repository (which is part of its key). The same manifest can be present in multiple repositories which in turn leads to "duplicate" manifest entries (aside from the fact they live in different repositories).

A similar concept holds true for layers of a manifest.

We have imported the container registry from dev.gitlab.org into a database using model 1. In this section, we gather statistics from that import to shed some light onto what to expect from this in-database deduplication.

The "dedup factor" is basically how much more entries we expect if we didn't deduplicate them.

| Entity   | Referenced from | Dedup factor |
|----------|-----------------|--------------|
| Manifest | Repository      | 1.003        |
| Blob     | Layer           | 1.53         |
| Blob     | Repository      | 1.17         |

Example for Manifest:

```
sql
select (select count() from repositorymanifests) / (select count()::numeric from manifests);
```

In the case of the registry on dev.gitlab.org, we can see that almost all manifests are unique (dedup factor 1.003). On average, blobs are being referenced by 1.53 layers and 1.17 repositories.

In summary, if we were not deduplicating those records in the database (model 2 below), we'd have less than double the number of records - assuming dev.gitlab.org is representative enough.

## Database design alternatives

### Model 1: "Many-to-Many" and no duplicate records

This approach has been validated to support all relevant container registry features. The database design discussion is well covered and includes query examples along with their plans.

The idea of this approach is to treat all entities as first-class citizens and use many-to-many reference tables to connect them. For example, for a given manifest digest, there'll ever be one entry in manifests. This entry might be referenced from multiple repositories through the many-to-many reference table repositorymanifests.

This naturally leads to deduplicating records, as they are not physically tied to the existence of the repository. See above "Deduplication Ratios".

## Benefits

Benefit 1: Well aligned with the existing filesystem model of container registry

## Drawbacks

Drawback 1: There is no common partitioning key

There is no natural partitioning key in this model. There are a couple of patterns how the tables are being access. Given all models are first-class citizens, there's no common dimension all tables (or a large enough subset of tables) are being accessed by.

This renders it problematic if not impossible to find a meaningful partitioning scheme. This has potential to lead to performance problems down the road, when tables become large.

Drawback 2: The model naturally leads to dangling records

The nature of this model is to treat all entities as first class citizens. This helps to deduplicate records, but also allows for a state where an entity doesn't have any references anymore but still exists in the database.

This means that we'll have to have a garbage collection algorithm to clean those entries up.

Drawback 3: Garbage collection is expensive

Since the model allows records to become "dangling", we'll have to implement a GC algorithm to find those and eventually delete them. Let's look at the example of dangling manifests: This is a manifest that is not being referenced by any repository. We can find a batch of up to N=100 dangling manifests like so:

```
sql
SELECT
FROM manifests m
WHERE NOT EXISTS (
  SELECT 1
  FROM repositorymanifests rm
  WHERE rm.manifestid = m.id
)
LIMIT 100
```

This is an anti-join across the full relation, which can be executed as a scan on two indexes. In the best case, we find N=100 records quickly. This is rather unlikely though since we generally strive to keep the registry clean of those dangling records. The worst case is the registry is clean (there are no dangling records). In this case, we scan the entire relation. This is expensive and non-linear due to the join-nature.

We have a good example for anti-join runtime characteristics. Here, runtime varies from a few milli-seconds (best case - many "dangling records" or "dangling records" at the beginning of the scan) to about 30s (no dangling records).

Drawback 4: Reference tables are expected to become large

The many-to-many reference tables are the ones that would become large, because they are effectively connecting N repositories with M manifests, for example (one record per connection).

Ultimately, this can become a performance problem when they become very large. This is particularly because the reference tables are being used in joins across one or more other tables.

Model 2: 1-N Repository structure with separate blob management

This approach treats a repository as the first-class model. A repository contains many manifests (1-N), those contain many layers (1-N). Separately from the repository structure, we keep track of blobs residing in object storage. We automatically maintain a reference table to keep track of which blob represents a given layer (lookup by blob digest).

Minor differences to model 1 include using a single digest column to store both algorithm and actual digest and normalizing the media type information into a lookup table.

Reference: SQL schema

What is not shown in the diagram is the possibility to have blobslayers being tracked automatically. This can be implemented in the database, but we may also do this from the application. The in-database implementation would rely on a trigger like so:

```
sql
CREATE FUNCTION public.trackblobusage() RETURNS trigger
    LANGUAGE plpgsql
    AS $$
BEGIN
    IF (TGOP = 'DELETE') THEN
        -- TODO: We can do more stuff here, this is just for illustrative purposes.
        -- Note: This doesn't have to be a trigger, it can also be application logic
        IF (SELECT COUNT() FROM blobslayers WHERE id <> OLD.id AND digest = OLD.digest AND layerid IS
NOT NULL) = 0 THEN
            INSERT INTO blobreviewqueue (digest) VALUES (OLD.digest) ON CONFLICT (digest) DO NOTHING;
        END IF;
    ELSIF (TGOP = 'INSERT') THEN
        INSERT INTO blobslayers (digest, repositoryid, layerid)
        VALUES (NEW.digest, NEW.repositoryid, NEW.id)
        ON CONFLICT (digest, layerid) DO NOTHING;

        INSERT INTO repositoryblobs (repositoryid, blobdigest)
        VALUES (NEW.repositoryid, NEW.digest)
        ON CONFLICT (repositoryid, blobdigest) DO NOTHING;
    END IF;

    RETURN NULL;

END
$$;

CREATE TRIGGER trackblobusagettrigger AFTER INSERT OR UPDATE OR DELETE ON public.layers FOR
ROW EXECUTE PROCEDURE public.trackblobusage();
```

In case of inserts to layers, we also keep track of the reverse lookup automatically in blobslayers and repositoryblobs . When we delete a layer, we can check if there are any remaining usages left for this blob (notice the efficient lookup by digest) and if not, we

might insert the digest into a queuing table. A background process takes those records and eventually cleans object storage and the blobs table.

Note this is just illustrative and we can get to the details later.

## Benefits

Benefit 1: Efficient garbage collection for blobs - no "scanning" GC needed

In contrast to model 1, this approach makes garbage collection straight forward. We don't ever need to scan entire tables to find "dangling" records. This is because the database contents are always consistent.

For example, when we delete a layer - we can determine the affected blobs easily and schedule those for further checking and eventually deletion. This can be implemented even inside the database (triggers), if we wanted to. If we rely on cascading foreign keys, all this can be triggered by a DELETE - even deleting a full repository may be possible (though we might want to consider batch deletes) this way, fully triggering a cleanup of all the manifests, layers and eventually scheduling relevant blobs for deletion.

Benefit 2: No GC needed for entities other than blobs

Blob management has a need for (some) GC algorithm, because we effectively deduplicate data in object storage. However, other entities like manifests and layers don't have a need to perform GC in this model.

This is in contrast to model 1 where we effectively allow a record to become "dangling" because we deduplicate all entities in the database, too.

Benefit 3: Supports partitioning by repository and digest (for blobs)

There are two distinct areas of the database, each with their own partitioning key:

1. A repository along with its structure (partitioned by repositoryid)
2. Blob management (partitioned by digest)

The choice of partitioning key dictates how the respective tables should be accessed. We must always use the respective partitioning key to make for most efficient queries.

For blob information, this leads to "Drawback 2" below.

Note that this partitioning scheme can ultimately also be used to create an application sharding design. We would apply the same idea for the repository structure (split by repository) and rely on a single blob management (or even divide that into more parts at the expense of decreasing space usage efficiency).

## Drawbacks

Drawback 1: Duplication of manifests and layers records

With repository being the top-level entity, manifests and their layers are not being deduplicated (in contrast to model 1, where we don't store a given manifest twice). This is intentional to support the ability to separate data by repository internally (see partitioning).

Since manifests contains the actual payload of a manifest, too, this may have a significant effect on overall database size. It remains to be seen if this is mitigated by partitioning. This depends on the efficiency of deduplicating manifests, i.e. what the dedup factor is.

#### Drawback 2: Duplication of blob meta-data in layers

A blob has meta-data attached like its digest, the size or the detected mediatypeid. This is stored in blobs but also in layers. The reason for this duplication is that we have queries that scan a range of manifests (e.g. all layers for a manifest). If we only had manifests.digest, we would have to lookup this information in blobs. This table in turn can only be queried efficiently by digest, rendering the lookups single-record queries and effectively a N+1 pattern. This is being mitigated by duplicating the information into layers.

#### Garbage Collection in Model 2

##### Examples of managing potentially dangling records

Note: In most cases, we will have to resolve a repository's <name> to its corresponding <id> first. This needed so we can use the id as a partitioning key in later queries.

##### Example: Delete manifest

text

```
DELETE /v2/<name>/manifests/<digest>
```

In this case, we delete the corresponding record in manifests:

sql

```
DELETE FROM manifests WHERE repositoryid=:id AND digest=:digest
```

The installed trigger inserts all referenced blobs (configuration and layers in the manifest) into the blobreviewqueue. A later GC process inspects those and deletes blobs in case they don't have any references anymore.

##### Example: Delete repository

Deleting a full repository doesn't seem implemented in the api specs. It works like deleting all manifests in the repository.

##### Example: Untagging a manifest

text

```
DELETE /v2/<name>/tags/reference/<reference>
```

In this case, we delete the tag:

text

```
DELETE FROM tags WHERE repositoryid=:id AND name=:reference
```

This can lead to untagged manifests which in turn are eligible for deletion. In order to defer this check to background GC, we insert all referenced blobs for the manifest that the tag points to into the blobreviewqueue, but only if no other tag points to the same manifest in that repository.

We might be able to skip this step early in case there is another tag for the same manifest. Note for simplicity, we can also do these checks in GC instead.

Example: Uploading a blob, pushing the manifest

text

```
PUT /v2/<name>/blobs/uploads/<uuid>
```

Once a blob upload is finished, we create the record in blobs and associate the blobs with the repository through repositoryblobs. In regular situations, the client might upload more blobs until it pushes the manifest. However, this process might fail and we would be left with dangling blobs (a configuration or a layer, which is related to the repository, but no manifest references it.)

In this case, we would insert the blob digest into the blobreviewqueue after the upload has finished (and before the corresponding records in blobs, repositoryblobs have been created). By setting blobreviewqueue.reviewafter to a time in the future, we can delay the checking of this blob (which effectively defines a timeout until when we expect the client to have pushed the manifest).

Now the client pushes the manifest.

text

```
PUT /v2/<name>/manifests/<reference>
```

We create the corresponding records in manifests and layers. After completing the verification steps, we can de-queue (delete) the digest from the blobreviewqueue. This is because we are sure now that there is a reference to this blob and we can skip GC. Note this is optional as the GC process would also be able to figure this out on its own - deleting from the queue is cheap (and likely cheaper than running GC), so this is an optimization.

Example: Switch existing tag from manifest A to manifest B

```
PUT /v2/<name>/manifests/<reference>
```

When uploading a manifest, if reference is a tag (can be a digest as well), we have to consider

the scenario where an existing tag is switched from manifest A to manifest B. For example, if we:

1. Build a docker image for a sample application, tag it with `myapp:latest` and push it to the registry. When pushing the image to the registry, its manifest, let's call it A, will be uploaded and tagged with `latest` inside the repository `myapp`;

1. Change the sample application source code, rebuild the image with the same `myapp:latest` tag and push it to the registry. Because we changed the source code, this image will have a different manifest, let's call it B, which will be uploaded with tag name `latest`.

When the registry receives the manifest B, it finds out that another manifest, A, is already tagged with `latest` in the repository. In this situation, the registry will remove the `latest` tag from A and point it to B instead. Because of this, manifest A may now be eligible for deletion if no other tag points to it.

To account for this we need to insert all blobs referenced by manifest A into the `blobreviewqueue`.

### Consuming the blob review queue

We implement a background job system that consumes entries from `blobreviewqueue` and performs GC checks and actions. This can be done with go-routines and synchronization on `blobreviewqueue` can be implemented with `SELECT ... FOR UPDATE SKIP LOCKED` mechanics.

Note: A particular choice here is that GC is performed on a single blob only. It can be expanded though to support batch GC. However, the database model being used here is optimized for making by-digest lookups.

### Popping the queue

In order to retrieve the next blob digest for review from the queue (the queue "head"), we find the next non-locked and qualifying record:

```
sql
BEGIN;
SELECT FROM blobreviewqueue WHERE reviewafter < NOW() FOR UPDATE SKIP LOCKED LIMIT 1;
-- now we perform checks and cleanup action
-- and once done, we remove the digest from the queue:
DELETE FROM blobreviewqueue WHERE digest = :digest;
COMMIT;
```

There is a caveat to that: In order to delete the blob from storage, we would perform external IO within an open database transaction (which is an anti-pattern). In order to mitigate this, we can do the following:

```
sql
BEGIN;
SELECT FROM blobreviewqueue WHERE reviewafter < NOW() FOR UPDATE SKIP LOCKED LIMIT 1;
```



```
UPDATE blobreviewqueue SET reviewafter = reviewafter + INTERVAL '5 minutes' WHERE digest = :digest;
COMMIT;
```

```
-- now we perform checks and cleanup action (outside scope of a database transaction)
-- and once done, we remove the digest from the queue:
DELETE FROM blobreviewqueue WHERE digest = :digest;
```

The trade-off here is that we have short database transactions but at the expense from changing the queue into having at least once semantics. That is, we can see multiple jobs trying to clean up the same digest. For example, when the external IO takes longer than 5 minutes (the offset for reviewafter) or a job fails completely. The assumption is that this is never a problem because the jobs are idempotent.

### Performing checks

Given a single blob digest, we implement a series of checks to determine whether or not the blob should be deleted from object storage:

1. Any remaining layer references? `SELECT 1 FROM bloblayers WHERE digest=:digest LIMIT 1`
2. Any remaining repository references? `SELECT 1 FROM repositoryblobs WHERE blobdigest=:digest` (TODO: The partitioning scheme is not suitable here, we might want to have blobsrepositories additionally - tbd)
3. Any remaining configuration references? `SELECT 1 FROM blobsconfigurations WHERE digest=:digest LIMIT 1`

The database model supports all those checks and we would typically benefit from the partitioning model plus using an efficient index (examples above: single record lookup).

### Performing cleanup action

Now that we've determined a blob should be deleted, we perform the following steps:

1. Delete blob from storage
2. Delete record in blobs and blobreviewqueue

We discussed a synchronization need with incoming requests. That is, an incoming request (like putting a manifest referencing this blob or checking) might have to be serialized with the cleanup action to get consistent results.

---

## SECTION: engineering/infrastructure-platforms/data-access/database-framework/doc/issue-group-search-partitioning/index.md

### Database Partitioning: Issue group search

We have motivated database partitioning by looking at a specific example: Group-based issue search.

This type of search allows to find specific issues within a GitLab group (example for gitlab-org group). We can apply all kinds of filters to the search, for example by filtering on milestone, author or a free-text search.

This document summarizes findings from exploring database partitioning for issue group search on a production-size dataset with PostgreSQL 11.

## Setup

We started from a copy of the production data of GitLab.com and upgraded the database to PostgreSQL 11 (to leverage partitioning features). This dataset now lives on a GCP instance comparable to a production database replica instance (in instance type and disk sizes).

In this database, we created a partitioned issues table with 64 partitions and employed hash-based partitioning by the top-level namespace the issue (and its project) resides in. This was done with several steps (SQL scripts):

1. Calculate top-level namespace information for each issue.
2. Create a partitioned table `parts2.issues` with the additional `toplevelnamespace` column to serve as the partitioning key.
3. Migrate and copy existing data from `public.issues` into this partitioned table.

## Query Analysis

Based on this setup, we looked at two types of queries:

1. Complex issue group search with normal filters and text search combined
2. Simple query to count issues for one project

Both cases are targeted towards a single group and are able to benefit from a partitioning scheme based on namespaces.

### Complex issue group search

Details and results for this query can be found in the issue. The properties of this example are:

1. Issue group search by milestone and text search for a logged in user within gitlab-org group
2. Targeted to a namespace and hence expected to greatly benefit from partitioning.

The results are:

1. We see a decrease in execution time in the order of about 8-10x, reducing warm-cache execution time from about 2s to 0.2s.
2. We see an increase in planning time from about 10ms to 40ms.

The increase in planning time is expected and it pays off as long as execution time is significantly improved.

### Simple count query

Details and results for this query can be found in the issue. The properties of this example are:

1. Simple query to count issues in a certain project (gitlab-org/gitlab)
2. Defaults to an index-only scan and does not read a lot of data
3. Given the table size of the non-partitioned case (about 20 GB), this query is expected to not show significant improvements from partitioning because it's not reading a lot of data anyways.

The results are:

1. With warm caches, total time is about 29ms and 50% worse in the partitioned case compared to non-partitioned.
2. The increase in total time is dominated by a significant increase in planning time for the partitioned case (from <1ms to 14ms).

#### Table and index sizes

As expected, individual sizes of partitions and their indexes are greatly reduced. Notably, indexes on partitions are between 10x and 100x smaller than on the non-partitioned table. This is where the main benefit from partitioning comes from: If we're able to statically exclude most of the partitions for a query, we're only going to deal with small indexes and small tables.

#### Planning times

We examined query planning times closer. While we've been doing query analysis earlier, we noticed an increase in planning times for the partitioned case. This is expected as the planner has to prune partitions and deal with meta-data and statistics for a lot more tables (each partition has their own).

We examined the planning time over a varying amount of attached partitions.

The benchmark ran 5 attempts for the same query after having re-organized partitions and attached the number of desired partitions to the table. We distinguish the first attempt from subsequent attempts due to caching behavior.

Furthermore we're looking at two cases here:

1. Query with partitioning key: WHERE projectid = ? AND rootnamespaceid = ?
2. Query without partitioning key: WHERE projectid = ?

The second case obviously doesn't benefit from partitioning during execution as it scans all partitions.

Starting with the simple query, this shows a rather expected result. As we can see the planning time depends on the number of attached partitions and increases slightly the more partitions we attach. The first attempt shows elevated planning times due to a cold cache (table metadata, statistics).

In the second example, we employ the same analysis but look at the complex issue group search example. This yielded quite unexpected planning times for the case without a partitioning key. This would drastically harm queries that don't have a partitioning key as a filter.

We suspected this might be due to gathering rather large statistics. On GitLab.com, we currently have `defaultstatisticstarget = 1000` which is 10x the default postgres setting. It directly controls the amount of detail the table histograms are going to have and therefore has a direct impact on the data that is relevant for query planning.

After dialing this down to `defaultstatisticstarget = 100` (the default setting), we arrive at more reasonable query timings. Luckily, this setting can be controlled on a per-table basis as well.

All data for these graphs can be found in a public sheet.

### Takeaways

1. Table partitioning helps to reduce execution times as expected.
2. It increases planning times for cases where most partitions get pruned. This is expected too and pays off when the subsequent execution time can be drastically reduced.
3. We need to be careful with queries that don't have a partitioning key.

### Queries without a partitioning key

Partitioning only works when queries contain a filter on the partitioning key. Only then the planner is able to exclude non-relevant partitions from the execution plan.

For queries without a partitioning key, it is important to know that partitioning also has an effect on their planning time. This can be a major problem as shown above and we might need to consider reducing statistic sizes for partitioned tables. We cannot expect queries without a partitioning key to simply perform at the same level as without partitioning.

This makes it harder to iteratively introduce table partitioning. Once we switch over to using the partitioned table instead of the original one, we risk suffering from queries without the partitioning key - perhaps even greatly as shown above.

This means we should make sure that - for a partitioned table - as many queries as possible benefit from the partitioning and naturally employ a partitioning key filter:

1. `activerecord-multi-tenant` can be used to derive the partitioning key automatically and add this filter when possible
2. We may want to find mechanics to detect and find queries early in the development cycle (i.e. on CI) that don't contain the partitioning key.
3. There are going to be cases where we can't use a partitioning key at all.

For example, finding issues assigned to a user doesn't come with a notion of a namespace but rather from a user's perspective. We will have to identify these cases and resolve them, for example by extracting their features into a separate service or duplicating data internally (e.g. the assignments table, to support both perspectives efficiently) and in some cases it might be possible to accept the increase in planning time.

Working page for PostgreSQL table partitioning

Also see the working page for Introducing PostgreSQL table partitioning to GitLab's database

---

Author: Andreas Brandl

---

## SECTION: engineering/infrastructure-platforms/data-access/database-framework/doc/multidb-bg-migrations/index.md

Background migration design for multiple databases

This is a working document to specify the design of background migration support for multiple databases.

Motivation

At GitLab, we rely heavily on background processing when migrating large volumes of data. This is not only important for typical data fixes related to application logic, but also as an underpinning for future database objectives, such as partitioning and schema redesign.

At the same time, we are decomposing the GitLab application database into multiple databases to scale GitLab. This effort has an impact on the entire application, requiring substantial changes to implement the desired design.

Since background migrations are such an important tool, we need to properly redesign them to fully embrace a multi-database design.

Background

Before we discuss the proposed design, we first describe how background migrations work, and the decisions imposed by the use of multiple databases.

Types of background migrations

We have two background migration frameworks within the application. Generally, the same multi-database design principles should apply to both.

1. Redis-based solution - These are background migrations generally available for use by any developer working on GitLab. While they do have optional job-tracking in the database, they primarily work by enqueueing sidekiq jobs to be processed on a fixed-interval delay.

1. Database-based solution - These are not generally available for use yet, but designed to be replacement for the redis-based solution. They run from a sidekiq-cron, but all job information is persisted in the database.

### Components of background migrations

We can think of background migrations as several loosely-coupled components:

1. Enqueuing migration helpers - called from a post-deployment migration, and store the information needed to later run the background jobs

1. Job execution framework - the generic worker process responsible for executing individual migration jobs, based on previously enqueued data

1. Background migration jobs - the jobs containing the specific business logic for the particular data migration

Since these components are logically separated, we don't have to apply a one-size-fits-all solution. The migration helpers run within a migration context, so they should naturally behave as Rails' migrations behave. The background migration execution is run from sidekiq, but the generic framework and custom classes have their own distinct concerns.

### Concerns of multiple databases

In a multi-database design, tables used to store data for business logic will reside only on a single database. The same set of business data will always come from the same known database.

Background migrations don't follow this model. Background migration jobs have a need to talk to multiple databases, potentially even at one time. Additionally, since both forms of background migrations have database-level tracking tables, we have to consider how we will divide this data across databases.

Separate from storage-level concerns, we need to consider our execution model. The use of multiple physical databases provides the opportunity to process data on each database independently.

### Proposed design

At a high level, we propose this design for multi-database background migrations:

- Enqueue migrations based on the intended context of the migration. Background migrations intended to execute in a particular context (e.g. "CI") are scheduled through Rails migrations for said context, and the tracking data is also stored on the database relevant in this context (e.g. "CI database"). If the background migrations are going to communicate with both databases then the developer can choose either ci or main based on which ever seems the most applicable.
- Use separate queues to process jobs for each database. The worker for that queue will know its own context, to connect to the appropriate tracking database.
- Allow maximum flexibility in individual migration jobs. Developers choose which databases to connect to execute their data migration.

The following diagram describes the high-level architecture of this approach:

We describe each of these points in more detail below.

#### Enqueueing migrations based on the intended context

For an example, with the decomposition effort, we intend to have a main database and a CI database. If a developer needs to run a background migration against tables on the main database, they will use a Rails migration for the main database to enqueue it. If they need to run the migration against tables on the CI database, it is enqueued by a Rails migration under CI.

This approach has several advantages:

- The migration naturally follows the model of Rails migrations. We don't have to reach cross-database (which Rails doesn't support) while the Rails migration enqueues the jobs.
- The execution framework can use SharedModel consistently both to access the tracking information and the database context for generic migration jobs which don't use a hardcoded connection to a particular database.
- We keep tracking data local to the database where the business data for the migration resides.

To support this execution model, we can require developers to tag migrations with the context they will need. A simplified example might look like:

```
ruby
class BackgroundMigration < GitlabMigration[1.0]
  restrictgitlabmigration schema: :ci

  def up
    schedule jobs
  end
end
```

This allows us to restrict the enqueueing migration to only run on the targeted logical database, unlike DDL changes, which will be applied to all database to keep the schema consistent.

#### Separate queues and workers

Again looking at a concrete example, if we have a main and CI database, we also have both a main and CI background worker.

Advantages to this approach:

- It provides a clean separation of concerns that is easier to reason about and maintain.
- It takes full advantage of multiple physical databases by processing against each independently. It also provides operation flexibility to manage these independently.
- The workers automatically know the context in which they run, without having to pass

additional data to them.

Allow the developer to choose how the migration connects

This is the only "decision" which is really a hard requirement. Individual background migration jobs might need to talk to one or more databases, so we have to leave the decision to the developer.

There are a couple ways this could work in practice depending on the scenario:

- The job uses migration-specific models that inherit from the correct base class for the intended database(s).

```
ruby
class ExplicitMigrationJob
  class SecurityScan < Gitlab::Database[:main]
  end

  class Build < Gitlab::Database[:ci]
  end

  def perform(startid, endid)
    buildids = SecurityScan.(id: startid..end).pluck(:buildid)

    Build.connection.execute(<<~SQL)
      DELETE FROM cibuilds
      WHERE id IN ({buildids.join(',')})
      AND (...)
    SQL
  end
end
```

- The job is designed to be generic and can use SharedModel (either as base class or to retrieve a connection) which is already configured by the execution framework for the correct context.

```
ruby
class GenericMigrationJob
  def perform(sourcetable, targettable, startid, endid)
    SharedModel.connection.execute(<<~SQL)
      INSERT INTO {targettable}
      SELECT
      FROM {sourcetable}
      WHERE {sourcetable}.id BETWEEN {startid} AND {end}
    SQL
  end
end
```



---

## SECTION: engineering/infrastructure-platforms/data-access/database-framework/doc/root-namespace-sharding/index.md

## Sharding GitLab by top-level namespace

This document summarizes the idea of sharding GitLab by top-level namespace. It is meant to provide an idea of how product features would have to change with namespace sharding and to highlight the difficulties and complexities we anticipate with this approach, as well as touching on implementation details and known unknowns.

Nomenclature - what is a namespace?

Let's start off by defining common nomenclature.

### Namespace

In GitLab, projects are contained inside a namespace.. A namespace can be nested inside another namespace. This forms a tree-like hierarchy: Namespaces are the nodes of the tree, projects are the leaves. The root of this hierarchy is called "top-level namespace".

Let's decompose this example: <https://gitlab.com/gitlab-org/database-team/team-tasks>

1. gitlab-org: top-level namespace
1. database-team: a group inside gitlab-org
1. team-tasks: a project inside database-team

After signup, each user has their own namespace associated under `gitlab.com/$user`. Additionally, a user may interact with any number of (root) namespaces across the instance.

### Data access pattern

A data access pattern is a commonly found perspective of how data is being looked at. This typically informs how database query filters look like. Let's look at a few examples:

1. Data by time: The primary query filter is perhaps a date range. GitLab example: Instance-wide audit log
2. Data by user: The primary query filter is the user id. GitLab example: User dashboard
3. Data by namespace: The primary query filter is the namespace id. GitLab example: issue group search

### Sharding by top-level namespace

The gist of this idea is to slice all data horizontally by the respective top-level namespace.

Let's use `/gitlab-org` and `/gitlab-com` as two example top-level namespaces. Both contain many sub-groups, projects, a lot of issues, merge requests, CI data and so on. With sharding applied at the database level, we separate those two namespaces in the database into separate shards (for the sake of the example):

1. Shard 1: /gitlab-org and all its data
2. Shard 2: /gitlab-com and all its data

Glossing over data distribution details for now, we can think of those two shards as physically separate databases.

Now, as desired, each shard is much smaller in data size compared to the non-sharded database. This results in much better performance for database queries that are designed around this sharding idea. More on that in: "Features in support of sharding".

On the other hand and strictly speaking, this limits product features in a sense that accessing any data has to always be scoped to a (root) namespace. Otherwise, we would have to scan all shards - which is undesirable and breaks at scale. So this is about access patterns and how some product features access the data. More on that in: "Features in conflict with sharding".

Complicating things, there is data that is completely unrelated to a namespace. We can see this by analyzing the database schema statically. Without even thinking about access patterns at this point, we can already conclude that this type of data is not shardable by namespace - it doesn't have any reference to a namespace.

Before we dive into details, let's summarize shortly what is ideal for this sharding approach:

1. There is only one common sharding dimension for related data, i.e. data that is commonly accessed together in one database query.
1. Shardable data has some notion of the sharding key, e.g. a reference to a namespace. This can be a transitive property, e.g. an issue does not have a direct reference to the top-level namespace but rather is contained in a project, perhaps other groups which eventually reference the top-level namespace.
1. Features are designed such that their data access patterns align with the sharding dimension.

The following section details apply these points to GitLab.

### Common sharding dimension in GitLab

We describe database sharding by top-level namespace and this builds on the premise of using the top-level namespace as the common sharding dimension throughout the application. Let's look into how well this works in GitLab.

### Database schema

GitLab's current database schema contains roughly 80 (out of 350) tables that do not have any notion of a namespace. This is a result from statically analyzing the database schema and basically checking for columns that (perhaps transitively) relate to a namespace. A summary for this analysis can be found in "A Note on the Technical Complexities with Sharding by Namespace" (GitLab-internal link).

In terms of data size, tables without a notion of namespace make up about 400 GB worth of data (about 6% of total size) as of today. Expecting a 100x growth, we would have 40TB worth of data that cannot be sharded. Depending on implementation details, this data would have to be

replicated across to expose it locally to each shard or we would only maintain this data on a central coordinator instance (which means we'd have to combine local and remote data from shards in queries).

Assuming we're not also adding capabilities for multi-master writes (i.e. the ability to write the same data in different locations), this hinders our ability to scale the read-write capacity - because these writes can only happen on a single cluster.

### Features and access patterns

A notion of the sharding key is required for sharding, but it's not sufficient to actually benefit from it. In order to leverage sharding, a feature needs to access sharded data by the sharding dimension. It is fine to add additional filters, but without the sharding key we are not only not making use of sharding but should expect a potentially severe degradation for query performance: Without the sharding key, we'd have to scan all shards (fan out) - which is very expensive. Depending on implementation details, it may be difficult to implement parallel cross shard scans.

Let's look at two examples here, more can be found in 50.

1. Issues: This is a representative example for data that originates from within a namespace, but also has conflicting access patterns.
2. Audit log: This is representative for features with conflicting access patterns at the instance level.

### Issues

An issue tracker lives inside a single project. As such, issue data transitively references a top-level namespace.

Sharding by namespace is ideal for these features:

1. Listing and working with a project issue tracker including search etc (example)
2. Similar the group level (example)

For these examples, we can clearly see the access path is "by namespace" or "by project" (which ultimately "by top-level namespace" covers).

On the other hand, there are features which would require a cross-shard scan:

1. User dashboard: Which issues are assigned to a particular user? Example
2. Similar: Instance-wide issue search by author, assignee, label, milestone, release, "my reaction". Example

These features have conflicting access patterns with the idea of sharding by namespace. This results in database queries that span all shards. Conceptually, this comes from the fact a user can interact with any number of namespaces across the instance. Therefore, in order to find assigned issues, we'd have to look into each and every namespace.

In a non-sharded database, this is easy: We maintain a single lookup table `issueassignees`

(userid int, issueid int) to model the N:M relationship.

In a database sharded by namespace, we would have two shards following the previous example:

1. Shard 1: /gitlab-org with a table issueassignees (userid int, issueid int)
2. Shard 2: /gitlab-com data with a table issueassignees (userid int, issueid int)

In order to find all assigned issues, we iterate all shards and query each of them. This is because we don't know which top-level namespaces the user interacted with at all and where to look for issue assignments (also see "Statistics on user interaction"). After querying all shards, we reconcile the results to typically display the TOP 20 issues (by some criteria) across all shards.

How can we fix this?

In the ideal scenario for namespace sharding, these features would be constrained by a namespace. This means, search cannot go across multiple namespaces. There are two theoretically possible ways to implement this:

1. Always let the user select a desired namespace for the search
2. On a more global level, constrain a single user to only be able to work and see data from inside a namespace at a time.

In both cases, it would be possible to change the currently active namespace. However, none of the features would display or work with data across multiple namespaces.

## Audit log

The audit log contains information about events that happened throughout the instance. An event may pertain to a user, a project or a group.

Based on that, we can already conclude that there are events which are unrelated to a project: User-related events do not have a notion of a namespace at all. On GitLab.com, this makes up about 75% of the audit log data, which cannot be sharded by namespace.

We allow users to work with the audit log from different perspectives:

1. By project
2. By group (which is behind a feature flag currently)
3. Global at the instance level (for admins)

The global instance level view is the problematic one: Ideally, an instance admin wants to see what happened across all namespaces and users on the instance.

In summary, if we sharded by namespace we would face:

1. 75% of data is not shardable in this dimension
2. Namespace sharding does not support the instance-level view

How can we fix this?

The audit log feature and its data is not suited well for namespace sharding in its current form unless we change the admin features.

For example, we could have an admin to always select a namespace of interest and we would only show data for a single namespace at a time.

However, in order to view audit logs for a user, we can only continue to build a large non-sharded table to provide this view. Depending on how isolated the audit log feature is from the rest of the application, we may want to think about feasibility of a different sharding dimension here (time or user).

GitLab.com: Statistics on user interaction

The premise of sharding by namespace is the idea that "customers care only about what happens inside their namespace".

For example, it may even be undesirable for a SaaS customer on GitLab.com to accidentally @-mention users outside their organization. A typical SaaS customer is assumed to only care about search results inside their namespace, for example.

In GitLab, we track which projects a user interacted with. Based on that, we conducted an analysis of how many projects, groups and top-level namespaces users interact with on GitLab.com. Here, interaction is defined as performing any kind of action on a project - opening and just looking at a project is not enough in this case (i.e. we're looking at actions that lead to an event). Note this includes users without any activity.

A summary and the takeaways are:

1. 95% of users interact with about 4 top-level namespaces or less (including their user namespace).
2. 60% of users only interact with 1 top-level namespace
3. 45% of users interact with a namespace other than their own
4. 15% of users interact with more than one namespace other than their own

More details including the raw data used can be found in [gitlab-org/database-team/user-interaction-data](https://gitlab-org/database-team/user-interaction-data).

While this is good news for the namespace sharding approach, it doesn't solve the fundamental problem of conflicting access patterns for all features.

For "by user" access patterns, we may be able to use this information to first figure out which shards are relevant for the particular user at hand and then only query those. This would help to avoid scans across all shards and limit it closer to what is relevant. However, this is not a static analysis anymore but rather involves maintaining lookup information to track which shards are relevant for a given user.

## Technical implementation of database sharding

GitLab is built on PostgreSQL which does not support native sharding (however, it's on the community roadmap). We explored using CitusDB instead, which is not an option for us.

We also explored using of PostgreSQL features such as partitioning in combination with foreign-data wrappers to implement a notion of a sharding database feature ourselves. While we identified a principal direction how this can be implemented, we are well aware of its complexities. This is not perceived as a "boring" solution and we do not know of positive reports from other companies going down this road. Also based on our own experience with using FDW for GitLab Geo, we deem this a very complicated approach.

Identified complexities reside around the following topics:

1. Operational: We would have to run multiple interconnected PostgreSQL clusters and a coordinator cluster on GitLab.com (each needs their own HA setup, coordinator is a single-point of failure).
2. Development: PostgreSQL FDW does not support parallel scans through FDW (fan out). All-shard scans seem infeasible at scale.
3. Development: Schema migrations are difficult to achieve in a distributed database.
4. Operational: Consistent backups of the database state are problematic.
5. Scalability: Write bottlenecks for tables replicated from a single point (for non-shardable data)
6. Product: Many product features have been identified to potentially become very slow/expensive or break with this approach. We would need to align the product thinking such that features are fully contained in a namespace. Ideally, this includes confining users into a single namespace, too - which seems like a huge undertaking and also quite limiting for the user experience.

We expect to only need sharding capabilities for GitLab.com and don't expect this to be necessary for self-managed installations. This further complicates implementation because it's likely we would have to implement some actions specific to GitLab.com.

## Summary

We explored what is needed for database sharding and we also got an idea of how well this would work with GitLab. In order to successfully implement namespace-based sharding, we identified two challenges:

1. Align product features with namespace-based sharding - it is expected that this requires a large amount of rethinking the product and design new features (and re-design existing ones) with namespace boundaries in mind.
2. Technical implementation of a distributed database backend - while we have ideas how this can look like, it seems like a risky and lengthy effort.

How does this relate to partitioning efforts?

We aim to introduce PostgreSQL table partitioning to the application. This can be thought of as a database tool we have at our disposal to considerably improve performance and scalability of the application.

It can be applied local to a single feature and optimize for a particular use case. For example, we are working on improving the audit log feature with a partitioning pattern. The benefit from this locality is the ability to optimize for different cases across the application - we don't have to find the single common sharding dimension for everything.

While we've explored partitioning to implement a notion of sharding (with FDW), partitioning is also simply a tool to improve database interaction. As such, it can be used additionally to sharding approaches: In a sharded database or in application sharding, partitioning is local to a shard and further improves database scalability.

In that sense, partitioning is a tool that is readily available for us to make use of now and one that does not introduce operational complexity. It directly improves performance and scalability of the database. On GitLab.com, we are at a scale currently where this seems to right tool to address performance issues. Additionally, it comes with the same fundamental thinking around slicing data horizontally and analyzing existing access patterns (and perhaps rethinking product features to align better).

Related links for partitioning:

1. On table partitioning
2. Table partitioning: Issue group search as an example
3. Database partitioning epic

Where to go from here

We are going to continue to explore ways to improve database scalability, particularly with GitLab.com in mind:

1. Systematic workload analysis, micro-analysis on a query basis and applying standard relational best practices for modeling, query building, indexing, partitioning etc - harvesting low hanging fruits (example).
2. A mid-term database capacity analysis is in progress to better inform necessity and timelines for scalability needs. A short term estimation is expected to yield first results, shortly.
3. Implement data retention strategies to reduce the size and growth of the database
4. Scale up seems possible still
  1. Read/write workload: We can still continue to scale vertically to improve read/write capacity on the primary if needed.
  2. Read workload: We continue to scale-out the read-workload across multiple replicas.

Aside from this, we continue to discuss sharding approaches. This is mostly a product question to start with, because it is rather unlikely this works out without needing product changes.

---

Author: Andreas Brandl

---

## SECTION: engineering/infrastructure-platforms/data-access/database-framework/doc/training-

content/batched-background-migrations.md

This is a placeholder document for our upcoming training article on batched background migrations.

---

## SECTION: engineering/infrastructure-platforms/data-access/database-framework/doc/training-content/index-performance.md

This is a placeholder article on how indexes impact performance on gitlab.com

---

## SECTION: engineering/infrastructure-platforms/data-access/database-framework/doc/training-content/introduction-to-database-review.md

This is a placeholder document for our upcoming training article on database review.

---

## SECTION: engineering/infrastructure-platforms/data-access/database-framework/doc/training-content/query-plans.md

This is a placeholder document for our upcoming training article on query plans.

---

## SECTION: engineering/infrastructure-platforms/data-access/database-framework/doc/workload-analysis/index.md

Workload Analysis for GitLab.com

This document discusses several approaches to understand the database workload for GitLab.com better. It aims to provide a few more perspectives on database workload, in addition to already existing monitoring solutions.

Index bloat

In previous studies, we've established that GitLab.com suffers a lot from bloat in btree indexes. That is, over time, some indexes tend to grow a lot beyond their ideal size - they take up more space than needed and become less efficient over time. The ideal size for an index is its most compact representation. This is the case when the index is built freshly, but regular updates to the index cannot maintain this compact representation over time in many cases.

For GitLab.com, we've previously introduced monitoring database bloat in good detail. This is based on a bloat heuristic for individual tables and indexes. Note this is an estimation only and the error is unbound - but it typically provides good insight into database bloat and is much cheaper than rebuilding indexes and tables fully (e.g. using VACUUM FULL) to determine



their individual level of bloat. Thanks to this estimation, we've exposed metrics available through Prometheus and best seen in this Grafana dashboard.

Aggregating those statistics, we can see that index bloat (yellow line) steadily grows over time while table bloat (green line) stays rather constant. The graph below shows about 3 months (early 2020) and an increase in index bloat from 240 GB to more than 600 GB.

A few months later, in summer 2020, overall index bloat has accumulated to more than 1.3 TB (see below). Notice that table bloat increase significantly, too - however that was due to a single table which a data migration dealt with during that time (so that was expected). Compared to the overall database size at that time of 8.5 TB, index bloat made up about 15% of the total size. This space is mostly wasted and it occupies memory.

We've started to address this through manual invocations of `pgrepack` in early September (the dip in index bloat and the vertical green lines indicate that). The overall impact of the autumn repacking efforts can be seen below. In multiple steps, we've successfully lowered index bloat to more acceptable levels - freeing up over 1 TB of space. After the data migration had finished, we also removed table bloat - again freeing up over 1 TB of additional space (the green line).

### Addressing index bloat long term

We came to realize that addressing index bloat is something we need to automate. This allows us to address this problem in a higher frequency, with no manual efforts and ultimately we maintain a healthy state of index bloat over time.

We also expect larger self-managed installations of GitLab to suffer from this problem. Therefore, we developed a reindexing feature that ships with GitLab. Currently, this is being rolled out to GitLab.com and once proven successful, the feature will be available and enabled in GitLab by default.

The reindexing implementation is based on the fact that a majority of index bloat stems from regular (non unique) btree indexes. Those can be recreated relatively easily, without risk of prolonged locking situations:

1. Create a new index with the same definition using a temporary name (index creation using `CONCURRENTLY` option)
2. Swap original index with newly created one
3. Drop the original index

Now this approach is not possible for indexes supporting primary keys. Fortunately, this is not where we need to deal with a lot of index bloat in any case - so we don't need to solve this problem right now.

Note that once GitLab requires PostgreSQL 12, we'll be able to leverage the concurrent reindexing feature introduced there. Furthermore, PostgreSQL 12 has seen significant improvements in terms of btree storage savings, which we expect to benefit from eventually, too.

In this cycle, we've enabled the feature on GitLab.com briefly. We've discovered an issue with functional indexes that led to an incident. The underlying problem is that a subset of functional indexes needs additional statistics. Those statistics get dropped with dropping the original index. Hence, we explicitly need to rebuild indexes before dropping the original index. This has led to an interesting conversation about the PostgreSQL 12 feature REINDEX CONCURRENTLY, which suffers from the same problem. A fix for the reindexing approach is being tracked in this issue.

The reindexing lives inside a Rake task that gets triggered through a cronjob. For GitLab.com, we expect to be able to maintain healthy levels of index bloat by running this task on weekends only, but ultimately we'd like to be able to run this job as a low-priority background task in a more frequent fashion.

Going forward and with 13.6, we are going to improve this feature by adding a good index selection strategy and observability features to it.

#### Relevant links

1. Automatic reindexing epic
2. Database bloat dashboard
3. Index bloat study (2019)

#### High resolution top-k query analysis based on pgstatstatements

In order to understand database workload in a good amount of detail, one can use tools like pgBadger to produce insightful reports. This analysis is based on the postgres log output and typically has two limitations:

1. Only covers a part of the workload: Postgres is configured to only log slow queries (logmindurationstatement = 1s on GitLab.com, so only  $\geq 1$ s runtime gets logged), faster queries are not part of the analysis but may contribute significantly to overall workload
2. Aggregates across the full time span: Timings and other metrics are typically measured and aggregated across the full timespan of the given log. This allows to understand the average workload: For example, we know what the most time consuming query was across the observation time (potentially days). This works well for rather homogeneous workloads, but falls short with rather spiky workloads - effectively hiding less frequent, but impactful queries.

As an example, let's consider an hourly cronjob that performs very expensive queries for a few minutes. It reads a lot of data from disk, invalidates most caches and causes high CPU. However, seeing those statistics aggregated across the whole day effectively hides this - not unlikely, other, more frequent queries dominate the statistics. What makes things worse here, those higher frequency queries are often less easily optimized.

How do we look at database workload? Let's say, we are concerned with the primary database instance and its workload. We may want to understand in more detail which queries retrieves

most records and which ones read the most data from disk (as an example).

With pgstatstatements, this information is readily available on a per-query basis for most queries ever executed on the database (there is a configurable threshold, currently we track pgstatstatements.max = 5,000 queries at most). This means the approach has much more detailed insight into the actual workload than a log-based approach, effectively addressing concern (1) from above.

### High resolution top-k approach

This approach deals with concern (2): It provides a way to keep track of any queries that were considered "offending", even if only for a very short period of time (think 30 seconds).

Let's introduce a couple things first and then outline how the analysis works.

**Snapshot and snapshot frequency:** The approach is based on taking snapshots of pgstatstatements at high frequency, e.g. at a snapshot interval of 30 seconds.

**Time frame and differential:** The time between two consecutive snapshots provides opportunity to calculate the differential between those snapshots regarding available metrics. For example, we understand the number of rows a certain query returned overall during this time.

**Offending query by a metric x:** A query is considered offending a particular metric x in a particular time frame, if it is among the top-k queries by x. As an example, we might look at the TOP10 queries by total duration during a certain time frame - those are considered offending.

**Offending strike count for a query:** We keep track of the overall number of times a query shows up as offending and call this the strike count.

### Methodology

The analysis performs a few steps:

1. Gather snapshots of pgstatstatements from a particular database host across an extended period of time with the desired frequency.
2. For each time frame, calculate offending queries for each relevant metric and keep track of those offenses.
3. For each relevant metric, provide a list of all queries that were offending (strike count > 0) during the total time.

In this approach, we still see queries in the result even if they were only being considered offending during one time frame (e.g. 30 seconds).

### Exemplary implementation and results

An proof of concept implementation is available in [gitlab-org/database-team/highres-stat-statements](https://gitlab-org/database-team/highres-stat-statements).

An example report can be found in this sheet ([GitLab internal link](#)). This is based on snapshots

taken off the GitLab.com primary during 32 minutes at a 30 seconds interval.

## pgstatstatements and Marginalia

Marginalia is a Ruby gem that annotates database queries with their origin. For example, for sidekiq jobs and web requests, we would see similar SQL comments alongside each query:

1. Sidekiq: /application:sidekiq,...,jid:...,jobclass:AuthorizedProjectUpdate::UserRefreshWithLowUrgencyWorker/
2. Web: /application:web,controller:issues,action:relatedbranches,correlationid:.../

This information is retained in pgstatstatements. We can use it to aggregate across sidekiq jobs, controllers or controller actions.

Following the same methodology as above but aggregating based on the "endpoint" (a sidekiq job class or controller action), we can find offending endpoints in a similar fashion.

This has also been implemented in [gitlab-org/database-team/highres-stat-statements](#). An example report based on the same data as above can be found in this sheet ([internal link](#)).

Worth to note that it looks like not all queries are being annotated by Marginalia sufficiently for this. There are some queries that only have application:web, but no further information. If we wanted to use this approach, we may want to make sure a good portion of queries has the right annotations.

## Using pgBadger

PgBadger can be used to generate aggregate reports based on log data. For GitLab.com, we can follow the [runbook](#) explaining more details.

As outlined above, this is based on log analysis and as such suffers from lack of detail (only slow queries are being logged). It is still useful to have this in the toolbox and provides a good way to reason about slow queries.

Example reports for the GitLab.com primary can be found [here](#) ([internal link](#)).

---

## SECTION: engineering/infrastructure-platforms/data-access/database-operations/\_index.md

## Mission

The mission of the Database Operations team at GitLab is to Build, Run, Own and Evolve the entire lifecycle of the PostgreSQL database engine for GitLab.com.

The team is focused on owning the reliability, scalability, performance & security of the database engine and its supporting services. The team should be seeking to build their services on top of Production Engineering::Foundations services and cloud vendor managed products, where appropriate, to reduce complexity, improve efficiency and deliver new capabilities quicker.

The team uses Engineering Principals to guide the decisions it makes for its services. The team does not explicitly have any self-hosted product responsibilities but we should contribute the lessons we learn running the database engine for GitLab at scale in production back to the Product, Development & Support teams to improve overall customer experience with GitLab, as well as collaborating with the Support teams when self-managed customers encounter complex database engine issues.

## Team Members

```
{{< team-by-manager-slug "rmar1" >}}
```

## Ownership

## Services

Systems and services we are primarily responsible for:

- PostgreSQL Core (VM)
- PostgreSQL Core (PGaaS) (coming soon)
- PostgreSQL High Availability and Load Balancing (e.g. Patroni, PGBouncer, consul, PostgreSQL Replication etc.)
- PostgreSQL Disaster Recovery (backup/restore and other techniques)
- Database Observability (Prometheus instrumentation, workload analysis etc.)